

Chapter 11

Measuring and Monitoring Velocity

As you recall from Chapter 9, “Planning a Release,” the release plan was created by breaking the project into a series of iterations, where each iteration included a certain number of story points. The number of story points completed in an iteration is known as the project’s velocity. When planning the project, we either used a known velocity (if we had one, perhaps from another similar project) or we made one up. Velocity can be a useful management tool, so it is important to look at the team’s velocity at the end of each iteration as well as during the iterations.

Measuring Velocity

Because velocity can be such an important measure, it is important to think about how we’ll measure it. Most stories are easy to count: The team completed them during the iteration so they are counted at full value. Assume, for example, that a team completed the stories shown in Table 11.1 during an iteration. As can be seen in this table, the team’s velocity is 23, which is the sum of the story points for the stories completed in the iteration. If the release plan assumed a velocity that is significantly different from 23, it may be necessary to reconsider the project plan. However, be careful about adjusting a release plan too early. Not only is an initial velocity prone to error, it can also be very volatile during early iterations. You may want to wait two or three iterations until you have a longer-term view of velocity.

Table 11.1 *Stories completed during an iteration.*

Story	Story Points
A user can...	4
A user can...	3
A user can...	5
A user can...	3
A user can...	2
A user can...	4
A user can...	2
Velocity	23

But, what about stories the team only partially completed? Should they be included in velocity calculations?

No, you should not include partially completed stories when calculating velocity. There are a number of reasons why. First, there's the natural difficulty of figuring out what percentage of a story is complete. Second, we don't want to imply a false precision to velocity by reporting it with fractional values like 43.8. Third, incomplete stories do not typically represent anything of value to users or customers. So, even though they may have been partially coded, such stories are often left out of formal end-of-iteration builds if the software is to be delivered to any users. Fourth, if stories are so big that including a partial story routinely affects velocity, say from 41 to 50, then the stories are too large. Finally, we desperately want to avoid a situation where many stories are 90% done, yet few are 100% done. A great deal of complexity can lurk in that last 10%, so it is important to finish each story completely before counting it.

If you find yourself tempted to include partially complete stories, evaluate the size of your average story and consider striving for smaller stories. At the end of an iteration when determining velocity, it is far easier to forego half of a one-point story than it is to ignore a twelve-point story. Additionally, if you frequently find iterations finishing with many partially complete stories (even if they are all half-point stories), this may be a symptom of a lack of teamwork on the team. With an all-for-one approach the team will learn they are better off joining together to complete some stories, leaving others unstarted, than they are partially completing all of them.

Velocity Does Not Use Actual Hours

Notice that velocity calculations are made using the story point values assigned before the start of the iteration. Once the iteration is complete, do not change the points the team earns for any story in the iteration. For example, suppose a story was estimated to be four story points but was much larger. After the fact, the team acknowledges they should have estimated it at seven points. This story contributes four points to the velocity calculation, not seven.

In general, teams are encouraged to plan the velocity of the next iteration to be no more than the velocity of the prior iteration. However, if the team is truly convinced that a story was estimated too low and they can do more in the upcoming iteration, then they should be allowed to plan on a slightly higher velocity.

While a team cannot retroactively change the points of a completed story, they should always make use of this type of information in adjusting the estimates of any future stories.

Planned and Actual Velocity

A good way to monitor whether actual velocity is deviating from planned velocity—and, more importantly, whether it's something you need to act on—is to graph planned and actual velocity for each iteration. This can be seen in Figure 11.1, which shows planned velocity starting low but then increasing and stabilizing by the third iteration.

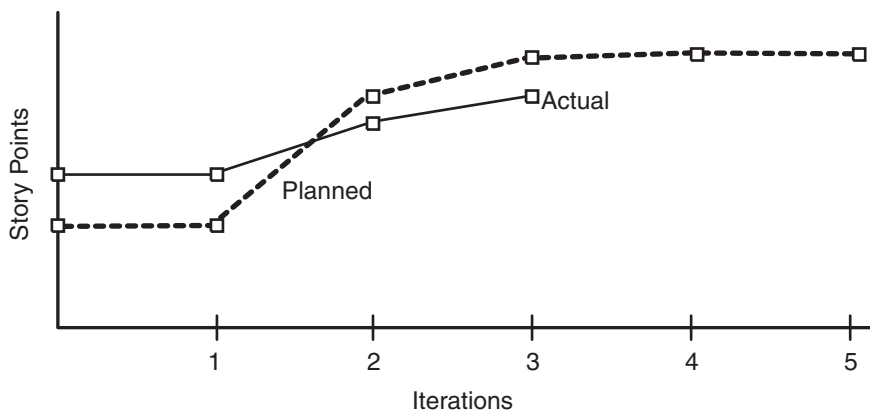


Figure 11.1 *Planned and actual velocity after the first three iterations.*

Actual velocity, graphed through the third iteration, exceeded planned velocity for the first iteration. However, the actual improvements during the second and third iterations were not as great as planned and so actual velocity is slightly less than planned velocity.

The team of Figure 11.1 would have been wrong if, at the conclusion of the first iteration, they told the customer they were exceeding planned velocity and could move up the delivery date. What about after the three iterations shown? Can the team tell if they should adjust the customer's expectations about the release plan? To answer that question the team needs both the velocity graph of Figure 11.1 as well as the cumulative story point graph of Figure 11.2.

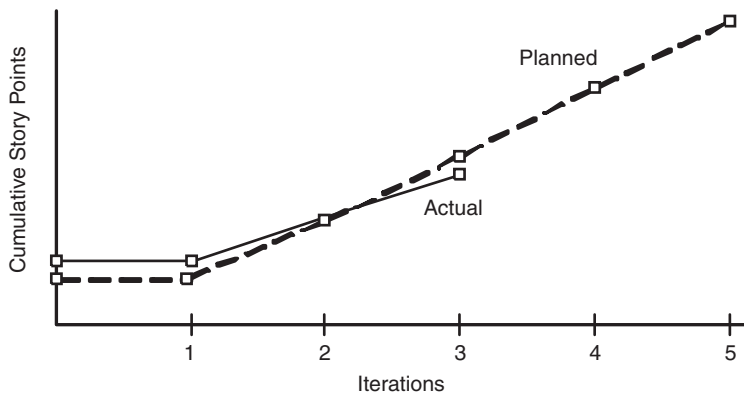


Figure 11.2 *Plotting cumulative planned and actual story points.*

The cumulative story point chart shows the total number of story points completed through the end of each iteration. So, we can see in Figure 11.2 that through the end of the second iteration the team had completed more story points than planned, even though progress in the second iteration was much slower than planned. However, by the end of the third iteration, the advantage of the team's good start in the first iteration has been eroded by slower progress during the second and third iterations.

By the end of the third iteration it appears probable that the team will not complete as much functionality as planned. If the customer is not aware of this from daily interaction with the team, the situation should be made clear to her.

Iteration Burndown Charts

Another useful way of looking at progress is through the use of an *iteration burndown chart*. An iteration burndown chart shows the amount of work, expressed in story points, remaining at the end of each iteration. An example is shown in Figure 11.3.

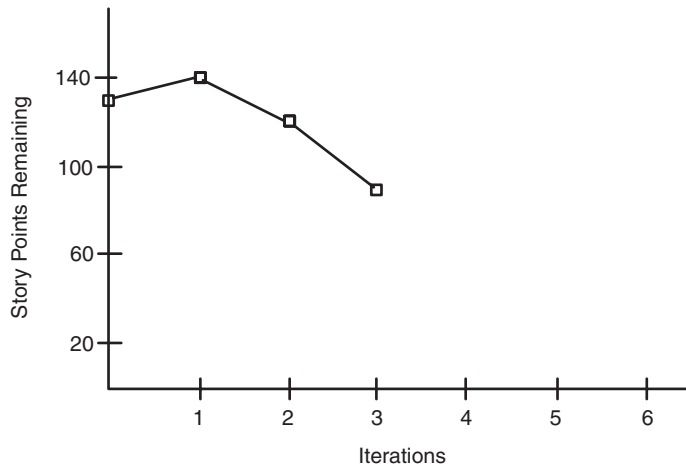


Figure 11.3 An iteration burndown chart.

An interesting feature of a burndown chart is that it reflects both progress made in the form of story points completed, as well as changes to the number of story points planned for the remainder of the release. For example, suppose a team completes twenty story points during an iteration, but the customer adds fifteen story points worth of new work to the project. The twenty story points reflects a net gain of only five; if the developers are working optimally then the customer may have to slow the introduction of new work if she expects the project to be finished quickly.

Figure 11.3 shows a team that actually had negative overall progress during the first iteration. They started the first iteration with 115 story points to complete and ended the iteration with 120. Managers and customers need to be careful that they don't read a burndown chart like Figure 11.3 and go yell at the team. What we can't tell from a burndown chart is how fast the team is moving. The team on Figure 11.3 may have completed 90 story points, but the customer may have added 95. To know how many story points the team is completing, look at a velocity chart (like the one shown in Figure 11.1) or a cumulative story point chart (like the one shown in Figure 11.2).

Burndown charts are useful—even though they don’t show the speed of the development team—because they present a better overall view of the project’s progress. A strength of agile software development is that a project can begin without a lengthy upfront complete specification of the project’s requirements. Agile teams acknowledge that it is impossible for customers to know everything in advance. So, agile teams instead ask customers to tell them as much as they can and allow customers to change or refine their opinions as the project progresses, and everyone learns more about the software being built. This means that stories will come and go, that stories will change size, and that stories will change in importance. As an example of this, consider the project shown in Table 11.2.

Table 11.2 *Progress and changes during four iterations.*

	Iteration 1	Iteration 2	Iteration 3	Iteration 4
Story points at start of iteration	130	113	78	31
Completed during iteration	45	47	48	31
Changed estimates	10	4	–3	
Story points from new stories	18	8	4	
Story points at end of iteration	113	78	31	0

The team on this project thought they could do around 45 story points each iteration. They started with 130 story points and a plan to run for three iterations. They completed exactly 45 story points in the first iteration. But, in completing those stories, they decided that a few of the remaining stories were bigger than initially thought and they increased the estimates on the unstarted stories by ten story points. Additionally, the customer wrote six new stories, each of which was estimated to be three story points. This means that while the team completed 45 story points, their net progress is 45–10–18, or 17 story points. This means that at the end of the first iteration they have 113 story points remaining. At this point the team could tell they would not finish in three iterations as planned. Even if no new stories came in, they had more than the 90 story points remaining, the amount they could reasonably expect to finish in the remaining two iterations. The customer and team met and considered stopping after three iterations—leaving some functionality out of the software—but, instead they agreed to let the project run for four iterations.

The second iteration followed a similar trend as the first, with the team completing 47 story points but increasing estimates of unstarted stories by 4 points. The customer slowed the pace of change but still added new stories worth 8

story points. Net progress during the second iteration was $47-4-8=35$ story points.

The team started the third iteration with 78 story points left. Things went well and the team completed stories worth 48 story points. They also reduced estimates of unstarted stories by three story points. (Remember that in the previous two iterations, estimates of unstarted work had gone up.) The customer added a story or two worth a total of four story points. Net progress during the third iteration was $48+3-4=47$. This left only 31 story points for the fourth iteration, which the team completed without any further changes.

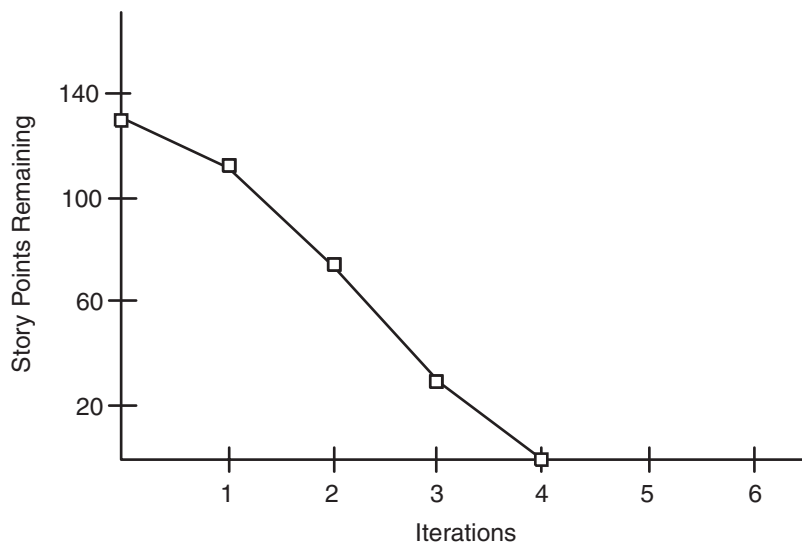


Figure 11.4 *Burndown chart for the project in Table 11.2.*

A burndown chart for this project is shown in Figure 11.4. As you can see from this chart, it was obvious from the slope of the burndown line after the first iteration that the project would not be finished after three iterations.

Burndown Charts During an Iteration

Beyond their usefulness for tracking progress at the end of iterations, burndown charts are also a great management tool during the iteration. During an iteration, a *daily burndown chart* shows the estimated number of hours left in the

iteration. For example, see Figure 11.5, which shows a daily tracking of the hours remaining in an iteration.

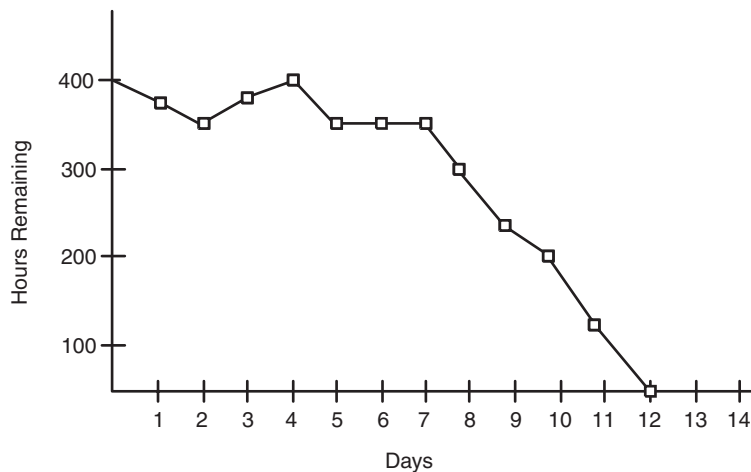


Figure 11.5 A daily burndown chart.

I prefer to collect information on the remaining level of effort by having each team member adjust his hours remaining on a common whiteboard. When iteration planning is done, the notes are left on the whiteboard that was used. At that point the board will contain a list of stories with each story disaggregated into one or more tasks. Next to each task is a place for the programmer to sign up for the tasks. About once a day I add up the numbers on the whiteboard and add them to a burndown chart for the iteration. Near the start of an iteration, a portion of such a whiteboard will look similar to Figure 11.6.

Create HTML page	Thad	2 0
Read search fields from HTML form	Thad	2
Create better sample data	Mary	2 4
Write servlet to perform search	Mary	4
Generate results page	Thad	2

Figure 11.6 Estimates are written, and frequently revised, on a whiteboard.

Figure 11.6 shows that the task “Create HTML page” has been completed as its estimate of remaining work has gone from two hours to none. However, Mary has increased her estimate for the “Create better sample data” task. It doesn’t matter whether Mary hasn’t started but changed her mind, or whether she’s already spent two (or four or six) hours on it and thinks there are four to go. All that matters is that the estimate on the board reflects her current thinking about how much work is left.

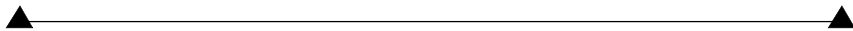
Everyone on the team knows to keep the numbers relatively current. It usually works best if the board is updated whenever a task is finished or at the end of the day. This way everything is always relatively current. Everyone on the team needs to be encouraged to have the estimates of remaining work be as accurate as possible. Programmers need to feel equally comfortable increasing an estimate of time remaining as they do decreasing it.



Hours Remaining, Not Hours Expended

Notice that daily burndown charts reflect the amount of work remaining, not the amount of work expended on a story or task. There may be a few good reasons for tracking hours expended (such as to compare actual to planned effort to improve estimating skills, or to monitor the number of productive hours spent each week). However, these are outweighed by the reasons not to capture hours expended (such as the feeling of being micromanaged most developers will feel and the effort or imprecision in the numbers).

Besides, what really matters is how much effort is left, not how much effort has been applied so far.



Naturally, new tasks can be added, too. However, a new task should be added only when someone realizes that a task was forgotten and the forgotten task is necessary for the completion of a story already included in the current iteration. New tasks should not be added just because someone wants something new included in the iteration; that type of change needs to be prioritized into an iteration at the next planning session.



Use Big, Visible Charts

All of the charts shown in this chapter are most useful when made big and visible. If your organization has a plotter then print the charts on the plotter and hang them on a wall in the team's common area or in a commonly used hallway. If you don't have a plotter, think about hanging a few large whiteboards on a wall and using those to draw the charts.

At one location we hung three large, four-foot by six-foot whiteboards in a common area. I got some small black tape at a stationery store and used it to draw axes on the whiteboards. This gave us permanently straight axes that weren't erased when we changed the chart. Each week we'd add a point to each of the three charts. Periodically, we'd run out of space so we'd erase the chart and start over.



Summary

- When determining velocity count only finished stories, that is, stories that pass their acceptance tests. Do not count stories the team partially completed during the iteration.
- A good way to monitor differences between actual and planned velocity is to graph both the number of story points planned and those actually completed for each iteration.
- Don't try to predict trends in velocity after only one or two iterations.
- The number of actual hours spent completing a task or a story have no bearing on velocity.
- Post big, visible charts in common areas where everyone can see them.
- A cumulative story point chart (as shown in Figure 11.2) is useful because it shows the total number of story points completed through the end of each iteration.
- An iteration burndown chart (as shown in Figure 11.3) shows both progress in the form of story points completed as well as changes to the number of story points planned for the remainder of the release.
- A daily burndown chart, showing the hours left on each day of an iteration, is very useful during an iteration.
- Charting and watching a team's defects per story point helps indicate if increases in velocity are coming at the expense of defects.

Developer Responsibilities

- To the extent possible, you are responsible for completing one story before moving onto the next story. It is preferable to have a small number of completed stories than to have a slightly larger number of stories all incomplete.
- You are responsible for understanding the impact of any decision you make on the velocity of the project.
- You are responsible for understanding how to read and interpret each of the charts shown in this chapter.
- If you are a manager or perhaps in the Tracker role on an XP project, you are responsible for knowing how and when to create the charts shown in this chapter.

Customer Responsibilities

- You are responsible for understanding how to read and interpret each of the charts shown in this chapter.
- You are responsible for knowing the velocity of the team.
- You are responsible for knowing how actual velocity compares to the planned velocity and whether plan corrections are needed.
- You are responsible for adding or removing stories from the release to ensure that the objectives of the project are met to the extent possible given its constraints.

Questions

- 11.1 A story estimated at one story point actually took two days to complete. How much does it contribute to velocity when calculated at the end of the iteration?
- 11.2 What can you learn from a daily burndown chart that you can't see on an iteration burndown chart?

- 11.3 What conclusions should you draw from Figure 11.7? Does the project look like it will finish ahead, behind or on schedule?

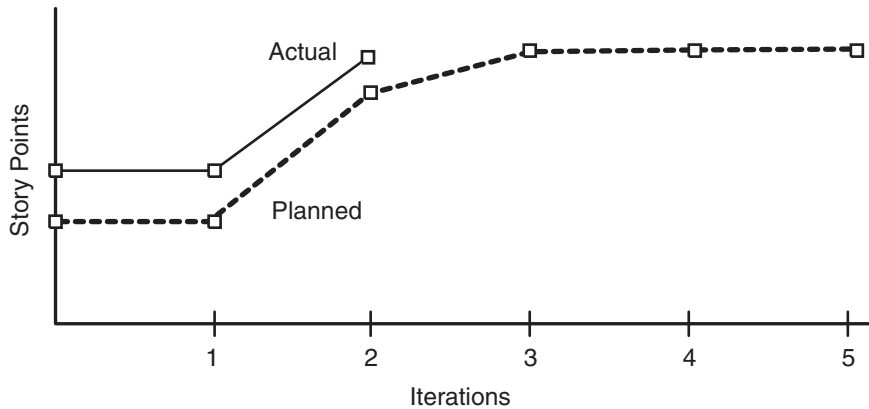


Figure 11.7 Will this project finish ahead, behind or on schedule?

- 11.4 What is the velocity of the team that finished the iteration shown in Table 11.3?

Table 11.3 Stories completed during an iteration.

Story	Story Points	Status
Story 1	4	Finished
Story 2	3	Finished
Story 3	5	Finished
Story 4	3	Half finished
Story 5	2	Finished
Story 6	4	Not started
Story 7	2	Finished
Velocity	23	

- 11.5 What circumstances would cause an iteration burndown chart to reflect an upward trend?

11.6 Complete Table 11.4 by writing the missing values into the table.

Table 11.4 *Fill in the missing values.*

	Iteration 1	Iteration 2	Iteration 3
Story points at start of iteration	100		
Completed during iteration	35	40	36
Changed estimates	5	−5	0
Story points from new stories	6	3	
Story points at end of iteration	76		0

This page intentionally left blank

PART III

Frequently Discussed Topics

We've looked at what stories are, how we can use them, and how we can estimate and plan using them. In Part III we start by turning our attention to how stories differ from other requirements approaches such as requirements specification documents, scenarios, and use cases. Next we look at the advantages user stories have over these other approaches.

As with any approach, however, things can sometimes go wrong and our attention turns next to a list of smells, or indicators that something may be amiss. User stories originated in, and are most closely associated with, Extreme Programming. In Part III we will see how user stories can be added to Scrum, another of the agile processes. Part III concludes with a discussion of some additional smaller but frequently discussed topics such as whether stories should be written on paper or stored electronically, whether stories should be written for bugs, and so on.

This page intentionally left blank